

ChangeGuard

Project Recovery Control Tower

A Microsoft Copilot Studio agent that turns an informal report of a slipping milestone into an assessed, approved and auditable recovery decision. The agent talks. Power Automate flows decide. A named person approves.

7

AGENT TOOLS

10

HARDENED FLOWS

11

DATVERSE TABLES

5

AUDIT EVENTS, ONE REQUEST

Everything here was produced on 22 September 2026 in an isolated Power Platform Developer environment, on synthetic data, and simulated figures are labelled as simulated wherever they appear. **Nothing in this document claims measured hours saved, cost savings, an actual freight booking, actual expenditure, a customer accepting a revised date, production deployment, fully proven concurrent execution safety, or zero cost operation.**

Item	Value
Agent	ChangeGuard Hardened Standard, Copilot Studio, Standard orchestration
Demonstration request	CR-CGDEMO-FINAL-01-CGDEMO-M1-2026-10-30-B1
Environment	ChangeGuard-Hardening-Test, an isolated Developer environment. Organisation identifiers are withheld from this document and available on request
Data	Synthetic. Project, milestones, calendar, recovery options and owners are invented for the demonstration

SECTION 1

Executive summary

ChangeGuard converts an informal milestone change report into an assessed, governed and auditable recovery process.

A supplier moves a delivery date. Someone has to work out which downstream commitments move, what the documented recovery choices are, what they cost, who may authorise them, and whether the agreed actions were carried out. That work is usually done in a spreadsheet and an email thread, and the reasoning is lost by the time anyone asks why the decision was made.

ChangeGuard captures the change as it is reported. A Copilot Studio agent reads the project baseline through a tool, computes the schedule impact deterministically in Power Automate, compares the documented recovery options as of a date, simulates the option the user is interested in, and saves a change request only after the user confirms the assessment in their own words. A reviewer then approves or rejects it in a Power Apps model-driven app, and only an approved request can be executed. Execution creates recovery work items with owners and due dates, and writes an audit event.

Intended users

- **Project managers and schedulers** who receive the change and need the downstream impact quickly.
- **A reviewer**, a PMO lead or delivery manager, who holds the authority to approve recovery spending or a revised commitment.
- **Delivery teams** who receive the recovery work items, and **auditors** who later need to see what was decided, by whom, and on what evidence.

Why governed recovery decisions matter, and where Microsoft AI fits

A recovery decision commits money or a customer date. If the calculation, the authority and the follow through are not connected, an organisation can only say what it believes happened. ChangeGuard makes the chain explicit: the supplied evidence, the computed impact, the selected option, the approver, the approval time and the work items that followed, tied together by one request key.

The language model is the interface, not the authority. Copilot Studio handles the conversation and decides which tool to call. Power Automate holds every rule: working day arithmetic, dependency propagation, option eligibility, approval enforcement and execution guards. Dataverse holds the records and, through alternate keys, enforces the uniqueness that makes a repeat safe. Power Apps is where a person, not an agent, takes the decision. The separation is testable: a refusal from a flow is a typed status, not a sentence the model composed.

The problem and why it matters commercially

Seven failure points sit between an informal supplier update and a defensible recovery decision.

Failure point	What goes wrong in practice
Informal supplier updates	The change arrives as an email or a sentence in a call. It is not a record, it has no structure, and the wording that justified everything afterwards is rarely kept.
Disconnected schedule assessment	The impact is recomputed by hand in a spreadsheet, usually by one person, using a working day rule that is not written down anywhere.
Inconsistent option evaluation	Recovery quotes expire. Options that were valid last week are offered this week because nobody re-checked the quote dates.
Unclear approval authority	It is often not obvious who is allowed to approve a recovery that costs money or moves a customer date, so approval happens in a reply that no system records.
Untracked recovery actions	The decision is made, then the follow up depends on someone remembering to raise the work.
Missing decision history	Months later the question is why the date moved. The answer exists in three mailboxes.
Duplicate execution	The same recovery is actioned twice by two people, producing duplicate work and, in a real deployment, duplicate spend.

Practical benefits, by role

Project managers

The downstream effect of a date change is computed from the baseline, the dependency graph and the working day calendar, in the conversation where the change was reported. The assessment is stored with the request.

Human approvers

The reviewer sees the original evidence, the computed assessment and the selected option on one read-only form, then decides. The decision is recorded with their identity and the time.

Delivery teams

An approved plan becomes recovery work items with owners and due dates, created by an authorised execution rather than by a reminder.

Auditors and boards

One request key returns the whole history: saved, refused while unapproved, approved, executed, and the safe repeat. Each event carries the run that produced it.

How each failure point is closed

Failure point	The control in ChangeGuard
Informal supplier updates	The user's own wording is stored on the request as the original evidence, and the save refuses unless the user confirms the assessment first.
Disconnected assessment	Dates are computed by a flow from the calendar table and the dependency closure, and the result is stored with the request.
Inconsistent option evaluation	One shared eligibility rule evaluates the documented catalogue as of a date, and the same rule runs again at execution time.
Unclear approval authority	The approver is checked against the project approver list before anything is written.
Untracked actions	An authorised execution creates the recovery work items with owners and due dates.
Missing decision history	Every transition writes an audit event carrying the actor, the states and the run that produced it.
Duplicate execution	Work items carry unique action keys, so a repeat reuses them and creates nothing.

Scope of the claim. ChangeGuard has been exercised on synthetic data in an isolated environment. This document contains no adoption metric, no return on investment, no hours saved and no cost saving, because none was measured. The benefits above are properties of the workflow, stated as such.

Solution overview

One conversation, four jobs: understand the change, compare the options, record a human decision, execute under guard.



Stage	Where it runs	Control that applies
Baseline	GetProjectBaseline flow, Dataverse read	The agent cannot proceed on a project code that does not exist. The tool returns a typed NotFound.
Assessment	AssessChange flow	Dates are computed from the calendar table and the precomputed dependency closure, never by the model.
Comparison	CompareRecoveryOptions flow	Each option is evaluated as of a date against its quote window. An expired or not yet quoted option is reported with the reason.
Simulation	SimulateRecoveryOption flow	The simulation states in its own response that it books nothing and changes no baseline.
Confirmation	Conversation	The save tool refuses with NotConfirmed unless the user has explicitly confirmed the assessment.
Save	SaveChangeRequest flow	The request row and its Saved audit event are written in one Dataverse changeset. A replay is caught by the request key alternate key.
Approval	Power Apps reviewer, Approve or Reject flow	The approver is checked against the project approver list. The decision and its audit event are written in one changeset.
Execution	ExecuteApprovedPlan flow	The flow re-reads the recorded decision and refuses unless it was an approval, by the recorded approver, for the option being executed.

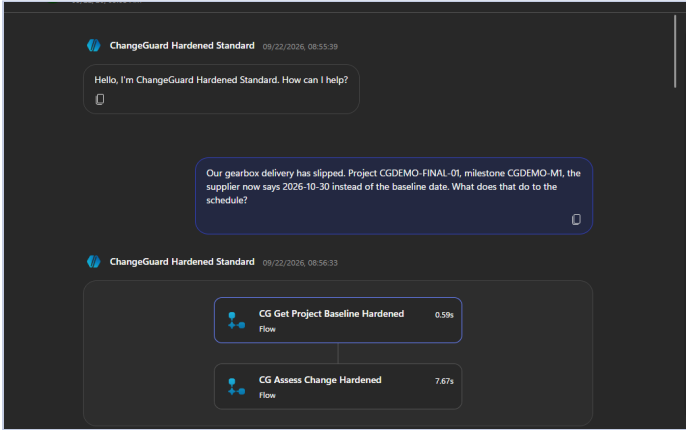


Figure 1. The conversation starts with an ordinary sentence. Copilot Studio calls CG Get Project Baseline Hardened and CG Assess Change Hardened, shown as tool cards with their durations. The user account name has been removed from this capture.

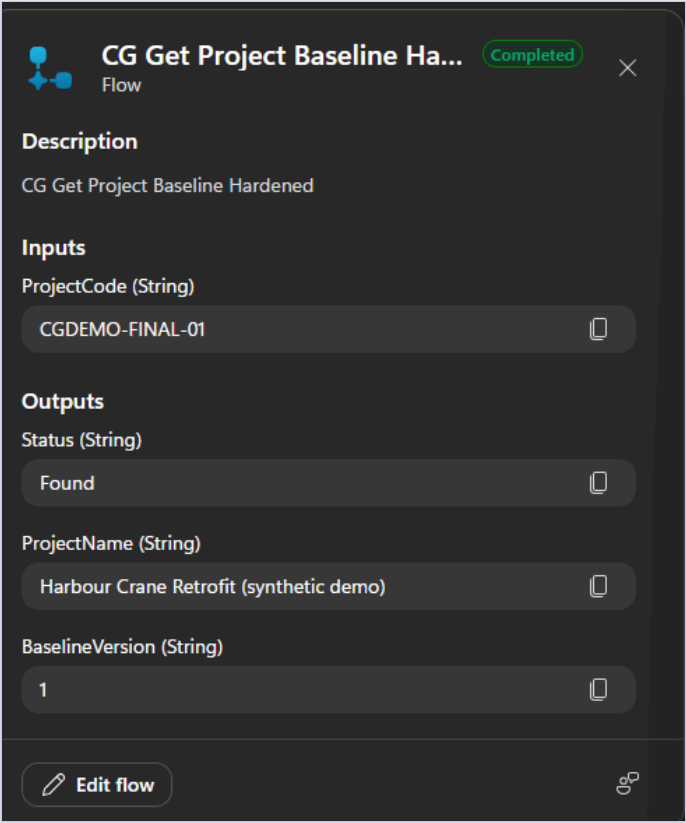


Figure 2. One tool call in detail: GetProjectBaseline returns Found, the project name and baseline version 1, read from Dataverse. Every number the agent states arrives this way.

Microsoft Copilot Studio

Standard orchestration, seven tools, and instructions written as a contract rather than a personality.

Why Standard orchestration

An autonomous agent was built first. Every turn was refused with an entitlement error, because the tenant holds no Copilot Credits from any source. A Standard, rule based agent runs the same Power Automate tools normally, so the shipped agent is the Standard one. The autonomous agent is deliberately kept in the environment as the evidence for that measurement. Orchestration choice, not environment type, is what drew the entitlement error.

Instructions and operational boundaries

The agent instructions are held in the repository, saved into the agent and verified by reading them back. They have six parts: identity, the data boundary, an eleven step workflow, human authority, the execution boundary, and audit traceability. In substance they forbid three things: inventing a value that no tool returned, claiming a save that did not happen, and treating the agent's own words as an approval.

The seven tools, each one a Power Automate flow

Tool	Flow	Responsibility	Writes
GetProjectBaseline	CG Get Project Baseline Hardened	Returns the project, its baseline version and its milestones	no
AssessChange	CG Assess Change Hardened	Computes the new finish date, the slip in working days and the affected milestones	no
CompareRecoveryOptions	CG Compare Recovery Options Hardened	Evaluates every documented option as of a date and explains each verdict	no
SimulateRecoveryOption	CG Simulate Recovery Option Hardened	Models one option in detail, including the downstream effect and the cost recorded in the catalogue	no
SaveChangeRequest	CG Save Change Request Hardened	Creates the change request and its Saved audit event after explicit confirmation	yes
GetRecoveryStatus	CG Get Recovery Status Hardened	Reports the request state, the recorded decision and the recovery work items	no
ExecuteApprovedPlan	CG Execute Hardened	Creates the recovery work items for an approved request, under guard	yes

Typed inputs, typed outputs, typed refusals

Each tool is invoked through a flow trigger with a declared input schema, and each flow answers with a declared response schema whose first field is a status. A refusal is a value in that contract, for example NotApproved, OptionNotEligible, DecisionConflict or AlreadyExecuted, so the agent reports a refusal rather than improvising an explanation. Every call leaves a flow run with its inputs, outputs and per action statuses, which is what makes the evidence in Section 9 checkable.

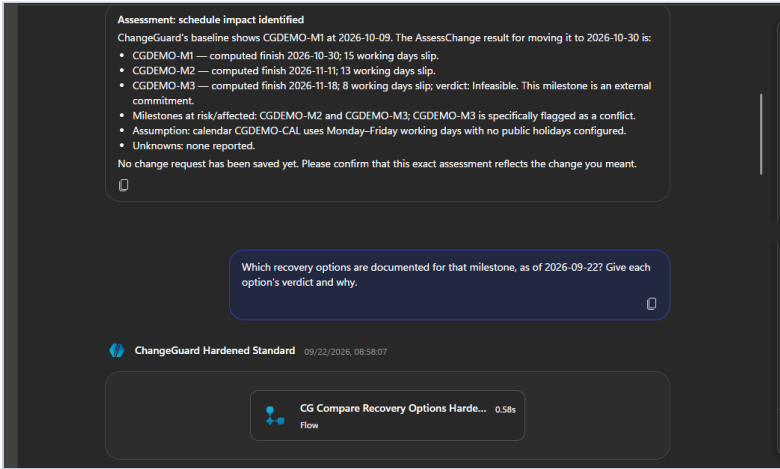


Figure 3. The assessment as the agent reports it: fifteen working days of slip on CGDEMO-M1, the two downstream milestones with their computed finishes, and the external commitment flagged Infeasible. The agent states that nothing has been saved yet and asks for confirmation.

Power Automate: where the rules live

Ten hardened flows are installed and active. Seven are agent tools, three are reviewer decision flows.

Flow	Responsibility	Entry point
CG Get Project Baseline Hardened	Project, baseline version and milestone set	Agent tool
CG Assess Change Hardened	Working day arithmetic over the calendar table, propagation through the precomputed closure, per milestone verdicts including the external commitment rule	Agent tool
CG Compare Recovery Options Hardened	Option eligibility as of a date: quoted from, valid until, availability status	Agent tool
CG Simulate Recovery Option Hardened	One option modelled against the plan, with the downstream effect and the catalogue cost	Agent tool
CG Save Change Request Hardened	Request persistence with the user's evidence, plus the Saved audit event in the same changeset	Agent tool
CG Get Recovery Status Hardened	State, recorded decision, work items and their open, overdue and blocked counts	Agent tool
CG Execute Hardened	Execution guards, work item creation, status transition and the execution audit event	Agent tool
CG Approve Hardened	Records an approval after checking the approver list	Reviewer app command bar
CG Reject Hardened	Records a rejection under the same checks	Reviewer app command bar
CG Decide Hardened	The same decision logic behind a direct trigger, for negative tests	Button trigger, testing

A platform behaviour worth recording. Copilot Studio exposes only a flow's *required* trigger inputs to an attached tool. The evidence text input on the save flow was optional, so the user's own wording was silently dropped until the trigger schema made it required. The fix was in the contract, not in the prompt.

Where each rule is enforced

- **Schedule arithmetic.** Working days come from a calendar table, not a date function, and milestones move through a precomputed dependency closure, so propagation is data rather than code paths.
- **Option eligibility.** One shared rule evaluates the catalogue: an option must be documented, belong to the project and milestone, and be inside its quote window on the date asked about. It runs again at execution time, as of the approval date, so a historic approval stays executable.
- **Approval enforcement.** The decision flows check the acting identity against the project approver list and otherwise refuse with DecisionNotAuthorized.
- **Execution guards.** Before creating anything, the execution flow re-reads the newest recorded decision and refuses unless it exists, was an approval, was made by the recorded approver and names the option being executed. The refusal statuses are NoRecordedDecision, DecisionNotApproved, ApproverMismatch, OptionMismatch, OptionNotDocumented, OptionWrongMilestone, OptionNotEligible, OptionNoLongerBookable.
- **Idempotency.** Work items carry a deterministic action key with a unique alternate key, so a repeat reuses the existing rows and creates nothing.
- **Error handling, fail closed.** If a Dataverse call fails, the run does not report success: an empty result becomes InternalError, the run terminates as Failed and the refusal is audited. This was added after a live run showed the opposite behaviour.

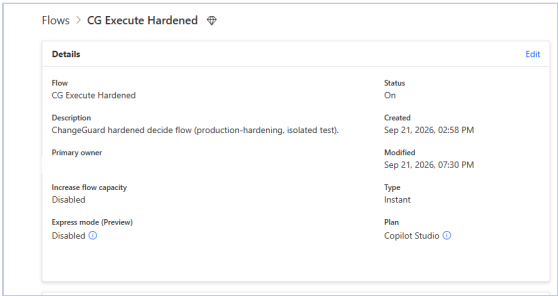


Figure 4. CG Execute Hardened in Power Automate, an Instant flow, switched On. The owner name and the platform estimated savings card were cropped out: that card is a platform estimate, not a ChangeGuard measurement.

Dataverse: the record and the enforced constraints

Eleven tables. Ten carry the workflow. Uniqueness is enforced by the platform, not by flow logic.

Baseline and reference

- **CG Project.** Project code, name, baseline version and the approver list used to authorise a decision.
- **CG Milestone.** Baseline finish, owner, and whether the milestone is an external commitment.
- **CG Dependency** and **CG Dependency Closure.** The dependency graph and its precomputed transitive closure, which is what makes propagation deterministic.
- **CG Calendar Day.** The working day calendar with a working day index.
- **CG Recovery Option.** The documented recovery catalogue: as of date, valid until, lead time, arrival date, cost and availability. This table is the trust anchor for what the agent may offer.

Governed records

- **CG Change Request.** The request, the original evidence, the assessment as returned by the tool, the selected option, the decision, the approver and the timestamps.
- **CG Recovery Action.** The recovery work items, with owner, due date and status.
- **CG Audit Event.** One row per state transition, with the event type, the old and new state, the actor and the detail.
- **CG Claim.** A short lived claim row with a lease, used to serialise decisions on one request.
- **CG Eval Result.** The eleventh table. It stores engine regression results per case and is not part of the request workflow.

Relationships

Milestones belong to a project. Dependencies and closures relate milestones to milestones. Recovery options hang off a project and a milestone. A change request references the project and the changed milestone, and carries the decision fields on itself rather than in a separate decision table. Recovery actions and audit events reference the change request. The claim row references the change request for the duration of a decision.

Alternate keys, and what each one guarantees

Alternate key	On	Guarantee
cr458_requestkeyunique	Change request key	A replayed save cannot create a second request. The flow reports AlreadyExists and writes one audit row.
cr458_eventkeyunique	Audit event key	An audit event cannot be written twice for the same transition and run.
cr458_actionkeyunique	Recovery action key	A repeated execution reuses the existing work items. This is the guarantee behind created 0, reused 2.
cr458_claimkeyunique	Decision claim key	Two simultaneous decisions cannot both hold the claim. The loser is refused with DecisionConflict.

Uniqueness is **server enforced**. The flows do not decide whether a duplicate is allowed: they attempt the write and handle the platform's refusal. A check performed in flow logic would be a race; an alternate key is not.

Atomic status and audit

The write tools commit their business row and their audit row inside one Dataverse changeset, so neither can exist without the other. The changeset has real constraints: its children may only create, update or delete, they run in parallel, and they cannot read one another's outputs. The audit payload is therefore computed before the scope opens, and the new row identifier is generated up front so that a sibling can bind to it. In execution, only the commit point is atomic: the work item creations sit in a loop and are reconciled by their action keys instead. That distinction is stated here rather than glossed over.

The Power Apps reviewer application

A model-driven app whose only job is to let a named person decide, on the evidence, without editing it.

The reviewer opens the change request in the ChangeGuard Reviewer Test app. The form shows the request details, the original supplier evidence in the user's own words, the baseline and proposed dates, the selected option, the assessment exactly as the AssessChange tool returned it, and the review status with the approver and timestamp. Every control is disabled, and the decision is taken from the record command bar by running CG Approve Hardened or CG Reject Hardened.

Why the form is read only

A form control is not a governance boundary. Read only fields stop accidental edits, but the reason the design is safe is different: execution re-reads the recorded decision from the audit trail and compares the option named there with the option being executed. An earlier build was proved vulnerable exactly here. The selected option was read only on the form, yet a direct write could still change it after approval, and execution acted on the tampered value. The guard, not the control, closed it.

Why approval must be a recorded human decision

- The approver is checked against the project's approver list before anything is written.
- The decision and its audit event are written together, so an approved request cannot exist without the event that explains it.
- The audit event carries the acting person, the requested decision and the option named at the time. Execution reads that event, not the request's status field.
- The rows themselves are written by the service principal the flow runs under, so the record separates who decided from what wrote the row.

Reported, not invented. The recovery option is stored as a code in a text column rather than a lookup, so the option's description, cost and eligibility cannot be shown on this form without a new column or a custom control. The slip in working days and the downstream verdicts live inside the assessment payload, so the form shows that payload rather than those values as labelled fields.

The screenshot shows the 'ChangeGuard Reviewer Test' app interface. The title bar indicates 'Power Apps | ChangeGuard Reviewer Test'. The left navigation pane shows 'Home', 'Recent', 'Pinned', and a 'Group' section with 'CG Change Requests'. The main content area displays a 'CG Change Request' for 'CGDEMO-FINAL-01 CGDEMO-M1 to 2026-10-30'. The 'Review' tab is active. The form is divided into several sections: 'Recovery decision' with 'Selected recovery option' set to 'CGDEMO-OPT-AIR' and 'Source' as 'ChangeGuard Control Tower agent (SaveChangeRequest)'; 'Assessment fingerprint' with a long alphanumeric string; 'Review status' with 'Decision' set to 'Approved', 'Approved at' as '9/22/2026 4:02 PM', and 'Approved by' as '[approver account redacted for publication]'; and 'Assessment detail as returned by the AssessChange tool' which contains a large JSON payload. At the bottom, the 'Record' section shows the 'Owner' as 'ChangeGuard-Automation'.

Figure 5. The reviewer form for the demonstration request after the decision. Selected recovery option CGDEMO-OPT-AIR, decision Approved, approval timestamp recorded, and the assessment detail exactly as the tool returned it. Every field is disabled. The approver account and the signed in user's avatar have been redacted for publication.

Governance and security

What is designed, what was tested by fault injection, and what remains open.

Identity and least privilege

Every flow binds a connection reference that resolves to a service principal connection, so all rows carry an application user as createdby and modifiedby. No flow carries an embedded personal connection and no flow contains an organisation URL. Four security roles are installed, CG Reviewer, CG Automation, CG Viewer and CG PMO Admin, covering eleven tables. A verification script read back **264 configured privileges** and matched them against the role design file with no over grants. The roles are installed but not assigned to any user in this environment.

Authorisation, in two independent places

- **At approval.** The acting identity must appear in the project approver list, otherwise the decision is refused with DecisionNotAuthorized and nothing is written.
- **At execution.** The flow re-reads the newest recorded decision and refuses unless it exists, was an approval, was made by the recorded approver, and names the option being executed. An approval is treated as an authorisation, not as a status flag.

Fail closed, proved by fault injection

The refusals above were not asserted from the definitions. One privilege at a time was removed from a security role, the flow was run, the behaviour was observed, the privilege was restored, and all 264 privileges were re-verified afterwards.

Privilege removed	Observed result
Read on the recovery option catalogue	The catalogue read was refused, the run ended Failed, an InternalError audit event was written, and no claim row was left behind.
Write on the change request	The claim was created, the update was refused, the run ended Failed with WriteFailed, and the claim was released. No DecisionRecorded event was written.
Create on the audit table	The changeset failed as a unit: the status update was skipped, the request stayed unchanged, no audit row appeared, and the run ended Failed.

The third case is the important one. Before the changeset was introduced, the same injection left a request approved with an approver set and no audit row at all. That defect was found by a live run, not by inspection, and the recorded evidence for it is deliberately kept.

Concurrency, stated precisely

The Dataverse connector exposes no conditional update and no ETag, so mutual exclusion cannot live in flow logic. A decision takes a claim row on a unique alternate key, with a lease. Two decisions fired together produce one DecisionRecorded and one DecisionConflict, which was reproduced live. An expired claim is reported as a stale claim and named, never stolen, because ownership transfer cannot be made safe without a compare and set primitive.

Not claimed. The claim row is not a compare and set. A holder that overruns its lease is not prevented from writing, so the lease narrows the double decision window rather than closing it. Closing it needs a pre-operation plug in or an ETag capable call. Live concurrent *execution* was not tested.

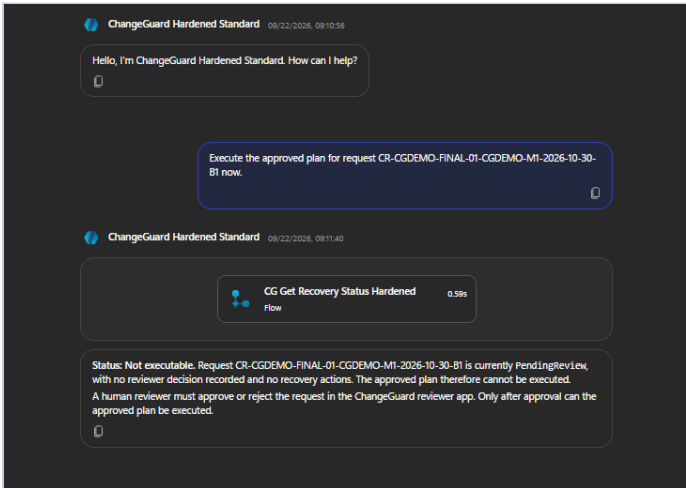


Figure 6. Authority in the conversation. Asked to execute the approved plan, the agent calls CG Get Recovery Status Hardened, finds PendingReview with no reviewer decision, and refuses. The flow-level refusal is independent of this one and is shown in Section 9.

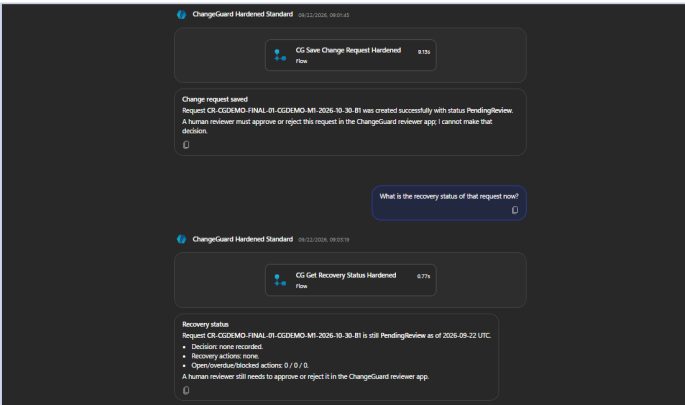


Figure 7. The save, then the status check. The agent names the request key it created, reports PendingReview, and states that it cannot make the decision itself.

The complete synthetic demonstration, part one

Harbour Crane Retrofit, project CGDEMO-FINAL-01. One request carried from a sentence to a saved record.

The baseline

Milestone	Baseline finish	Note
CGDEMO-M1, gearbox delivery to site	9 Oct 2026	the milestone that slips
CGDEMO-M2, mechanical installation	23 Oct 2026	depends on M1
CGDEMO-M3, commissioning and handover	6 Nov 2026	external commitment

The reported change

The user writes, in their own words, that the gearbox delivery has slipped and that the supplier now says 30 October 2026 instead of the baseline date. The calendar is Monday to Friday with no public holidays configured.

Assessment, computed by the flow

Milestone	Without recovery	Slip	Verdict
CGDEMO-M1	30 Oct 2026	15 working days	Changed
CGDEMO-M2	11 Nov 2026	13 working days	Slips
CGDEMO-M3	18 Nov 2026	8 working days	Infeasible, it is an external commitment

Documented recovery options, evaluated as of 22 September 2026

Option	Verdict	Documented arrival	Reason given by the flow
CGDEMO-OPT-AIR, air freight	Selectable	16 Oct 2026	confirmed and inside its quote window, cost recorded as 8,200 USD
CGDEMO-OPT-PART, partial airlift	NotYetQuoted	21 Oct 2026	provisional, not quoted until 5 Oct 2026
CGDEMO-OPT-SEA, sea freight re-quote	Expired	30 Oct 2026	validity ended 18 Sep 2026, before the scenario date

Simulation of the selectable option

With AIR, the simulation returns M1 on 16 October, five working days late, M2 on 28 October, three working days late, and M3 on 6 November with no date impact on the external commitment, at a recorded incremental cost of 8,200 USD. The response states in its own words that this is a simulation only and that it does not book freight, spend money or change the project baseline.

The distinction that must survive every retelling. The preserved customer date is a *simulated* outcome of a documented option. No freight was booked, no money was spent, no baseline was changed and no customer agreed to anything. When the plan was later executed, what was created was internal work, including a work item to agree the revised date with the customer.

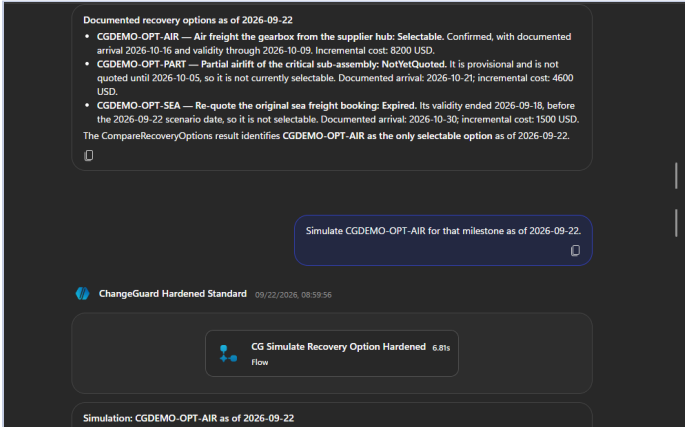


Figure 8. Option comparison as of 22 September 2026, as the agent reports it. AIR Selectable, PART NotYetQuoted until 5 October, SEA Expired on 18 September. The verdicts and the reasons come from the flow.

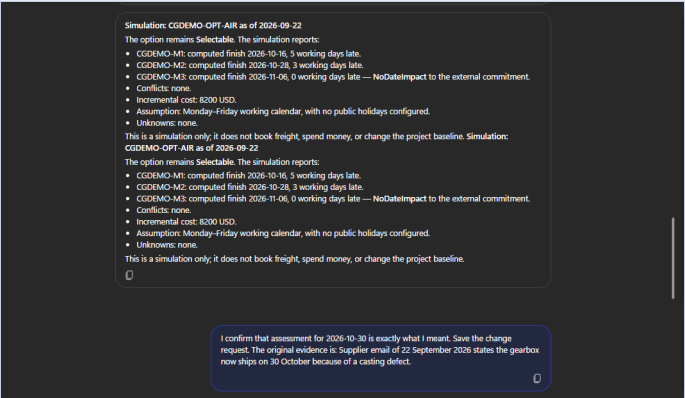


Figure 9. The AIR simulation, then the user's explicit confirmation and their own evidence text, a supplier email citing a casting defect. Nothing is saved without that confirmation.

From saved request to executed plan

Every row below carries the flow run that produced it, so the evidence can be opened rather than believed.

Stage	Flow run identifier	Result
Save	08584115247919949644217097079CU03	Created, request key CR-CGDEMO-FINAL-01-CGDEMO-M1-2026-10-30-B1, status PendingReview, evidence stored
Status	08584115246893388639659212600CU00	PendingReview, no decision recorded, no recovery actions
Refusal, conversation	08584115241882836559484613175CU07	The agent checked the status and refused to execute. It did not call the execution tool
Refusal, flow	08584115238858633865401322471CU27	The execution flow was run directly with the same request key and refused on its own: ExecuteNotApproved, created 0, reused 0
Human decision	08584115139116571959138843143CU31	PendingReview to Approved, approver recorded, one DecisionRecorded event naming CGDEMO-OPT-AIR
Execution	08584115135580615032139159456CU20	Executed, created 2, reused 0, status Approved to Executed
Repeat execution	08584115132348176988244243197cU14	AlreadyExecuted, created 0, reused 2, work item count unchanged

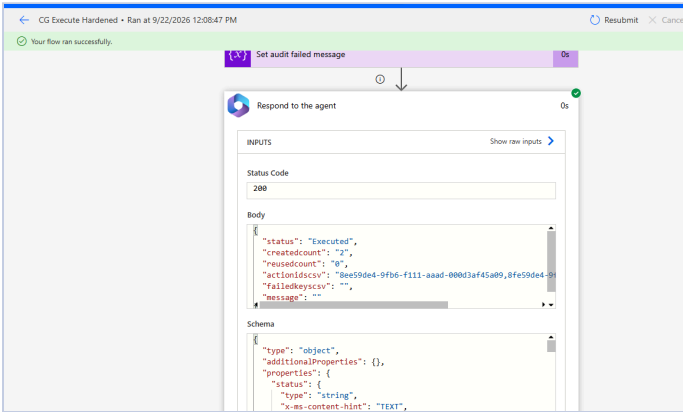


Figure 10. First execution, 12:08 PM. The flow's own response body: status Executed, createdcount 2, reusedcount 0, no failed keys. The green run banner alone is not the evidence.

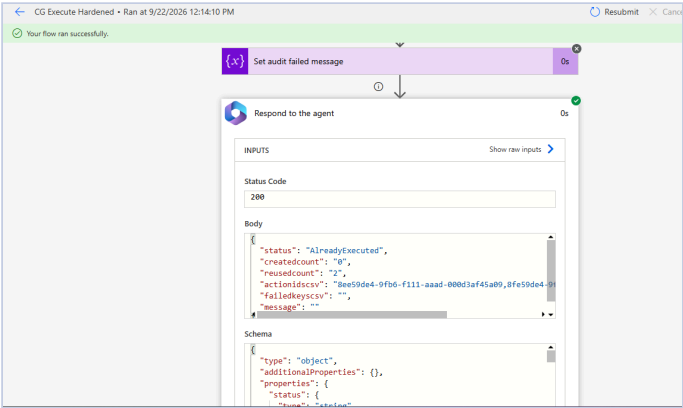


Figure 11. Repeat execution, 12:14 PM. Status AlreadyExecuted, createdcount 0, reusedcount 2. The action key alternate key, not the flow logic, is what makes this safe.

The two recovery work items created by the authorised execution

Key	Work item	Owner	Due	Status
...-B1-1	Book recovery option CGDEMO-OPT-AIR for CGDEMO-M1	Demo Owner One	9 Oct 2026	Open
...-B1-2	Agree revised CGDEMO-M3 commissioning and handover date with the customer (computed 18 Nov 2026)	Demo Owner Three	6 Nov 2026	Open

The first due date is the air freight quote's own expiry, so the booking task cannot silently outlive the quote it depends on. Both items are internal work. Nothing was ordered from any supplier.

Testing and verification

Four kinds of evidence, kept apart: live runs, fault injection, offline contract checks, and a validation matrix from the earlier build.

Every flow definition is produced by Python builders held in the repository, and the hardening sits behind flags that default to off, which is how the original release definitions still regenerate byte for byte while the hardened variants carry the extra guards. Each deployed definition was read back and compared with its source. That is what the first suite below checks.

Offline checks, re-run on 22 September 2026 for this document

Suite	Result	What it proves
Hardened flow contract checks	709 of 709	The generated hardened definitions carry the guards, the fail closed wiring and the deploy target split
Assessment engine parity	18 of 18	The working day and dependency arithmetic matches the expected results case by case
Approval guard generator	37 of 37	The approval time validation behaves as specified and stays off by default
Security model checks	28 of 28	The role design contains no over grant and matches the documented model
Submission utility checks	22 of 22	The packaging and privacy tooling behaves as specified
Flow structure checks	PASS, 33 definitions	Nesting stays inside the platform limit of eight, every runAfter target exists, and no definition contains an organisation URL

Live evidence in the hardened environment

- Six agent initiated tool runs for one request, each one Succeeded, each result read back from Dataverse.
- Save refused without confirmation, and a replayed save reported AlreadyExists without writing a second request.
- Execution refused before approval at both levels, once by the agent and once by the flow. The flow level refusal is the one that appears in the audit trail.
- A human approval taken in the reviewer app, recorded with the approver and the time.
- Execution created two work items, and the repeat created none and reused both.
- Two simultaneous decisions produced one DecisionRecorded and one DecisionConflict.
- An unauthorised approver was refused after the approver list was repointed to a fictional address, which was then restored.
- Fault injection on three separate privileges produced three fail closed outcomes, after which all 264 privileges were re-verified.

The validation matrix, and which build it describes

A 151 case validation matrix exists for the **original demonstration build**, with 145 PASS, 3 PARTIAL, 0 FAIL, 1 BLOCKED and 2 NOT RUN. The counts were recounted from the workbook for this document. That matrix is not a test record of the hardened build described here, and it is reported separately for that reason.

CG Audit Events				
Name	Created On	RequestKey	CG Change Request	
Saved CR-CGDEMO-FINAL-01-CG...	9/22/2026 9:01 AM	CR-CGDEMO-FINAL-01-CGDEMO...	CGDEMO-FINAL-01 CGDEMO-M1...	
ExecuteNotApproved CR-CGDEM...	9/22/2026 9:16 AM	CR-CGDEMO-FINAL-01-CGDEMO...	CGDEMO-FINAL-01 CGDEMO-M1...	
DecisionRecorded CR-CGDEMO-F...	9/22/2026 12:02 PM	CR-CGDEMO-FINAL-01-CGDEMO...	CGDEMO-FINAL-01 CGDEMO-M1...	
ExecuteExecuted CR-CGDEMO-FL...	9/22/2026 12:08 PM	CR-CGDEMO-FINAL-01-CGDEMO...	CGDEMO-FINAL-01 CGDEMO-M1...	
ExecuteAlreadyExecuted CR-CGD...	9/22/2026 12:14 PM	CR-CGDEMO-FINAL-01-CGDEMO...	CGDEMO-FINAL-01 CGDEMO-M1...	
Enter text		Enter text	Select lookup	

Figure 12. The five audit events for the demonstration request, in Dataverse, sorted by Created On ascending: Saved, ExecuteNotApproved, DecisionRecorded, ExecuteExecuted, ExecuteAlreadyExecuted. The trail contains one refusal event, written by the flow. The agent's conversational refusal is a separate control and did not write an audit row.

Known limitations

Stated in the same document as the claims, because a control that has not been tested is not a control that has been proved.

Limitation	Statement
Isolated environment, synthetic data	Everything runs in one isolated Developer environment on invented projects, milestones, owners and options. No real supplier, customer or commitment is involved, and the exported solution has not been imported anywhere else, so a clean import is unverified.
No production deployment	There is no customer deployment, no enterprise certification and no independent security review.
Limited human identity testing	The tenant has one enabled human identity, so an unauthorised second person could not attempt an approval. The rule was exercised by changing the approver list, which tests the rule but not a second person.
Live concurrency	Concurrent decisions were tested and refused correctly. Concurrent <i>execution</i> was not tested live.
Decision claim	The claim row is a unique key plus a lease, not a compare and set. A holder that overruns its lease is not prevented from writing, and an expired claim is reported rather than reclaimed. Clearing a dead holder is an operator action.
Execution atomicity	Only the status transition and the execution audit event are atomic. Work item creation is reconciled by unique action keys, not by a transaction, so a partial execution is reported as PartiallyExecuted and completed by a retry.
Conversation correlation	No conversation identifier is stored; the request key is the correlation identifier.
Metered usage and cost	Usage is attributed to a flow while purchased, assigned and consumed capacity read zero. The invoiced amount is unknown, so this is not called free.
Reviewer form coverage	The option code is a text column rather than a lookup, so its cost and eligibility are not labelled fields on the form; the computed slip and verdicts live inside the assessment payload the form displays.

Conclusion

What this build demonstrates about combining an AI interface with deterministic, human governed Microsoft Power Platform workflows.

ChangeGuard is not a chat wrapper over a database. The conversation is the interface because the intake is unstructured language; everything after it is a rule, so the rules live in flows and the records in Dataverse. That division is what makes each claim here checkable: a number traces to the run that produced it, a refusal is a typed status rather than a sentence, and an approval is an audited event that execution re-reads.

The defects found along the way are the strongest evidence for that division. An approved request could once exist without its audit row; a rejection once satisfied an execution guard that only looked for a recorded decision; a tampered option code once reached execution although the form showed it read only. Each was found by running the system, and each was closed in the flow layer, where the platform enforces the outcome, rather than in the prompt, where it cannot.

What a reviewer can verify without a tenant

- The flow definitions are generated from source, so they can be regenerated and compared, and the offline suites in Section 10 run with no network, no credentials and no Power Platform licence.
- The exported unmanaged solution carries the tables, the flows and the agent, with the target organisation as an environment variable and the data connection as a connection reference, so nothing is hard coded to the environment it was built in.

Where it would go next

- A pre-operation plug in or an ETag capable call, to make the decision claim a true compare and set.
- A clean import into a second environment, and a second human identity, so that portability and an unauthorised approver are proved rather than argued.
- A lookup column for the recovery option, so the reviewer form can show its cost and eligibility as fields.

The sentence that carries the design: the agent talks, the flows decide, a person approves. The work was in making it enforceable rather than aspirational.